

Lab 07 Collections (*SUBMIT ALL YOUR CODE ON MOODLE*)

1. [20 points] **DynamicArray Package**

Java's ArrayList is backed by a plain array that **automatically resizes** when it runs out of capacity. Your task is to build a simplified version of this from scratch.

Part 1 - Core Structure: Create a generic class `DynamicArray<T>` with:

- A private `Object[]` internal array
- A private `int` size to track the number of elements
- A default initial capacity of **10**
- Two constructors:
 - `DynamicArray()` - default capacity
 - `DynamicArray(int initialCapacity)` - custom capacity

Part 2 - Required Methods

Implement **all** of the following methods:

Method

`void add(T element)`
`void add(int index, T element)`
`T get(int index)`
`T set(int index, T element)`
`T remove(int index)`
`boolean remove(Object o)`
`int size()`
`boolean isEmpty()`
`boolean contains(Object o)`
`int indexOf(Object o)`
`int lastIndexOf(Object o)`
`void clear()`
`Object[] toArray()`
`void ensureCapacity(int minCapacity)`
`void trimToSize()`

Description

Appends element to the end
Inserts element at given index
Returns element at index
Replaces element, returns old value
Removes & returns element at index
Removes first occurrence of object
Returns number of elements
Returns true if size == 0
Returns true if element exists
Returns first index of element, or -1
Returns last index of element, or -1
Removes all elements
Returns a copy of the internal array
Grows array if needed
Shrinks internal array to current size

Part 3 - Resizing Logic

- When add() is called and the array is **full**, resize to **1.5× or 2× the current capacity**
- Use Arrays.copyOf() or manual copying - **do not use ArrayList or any Collection internally**
- Implement a private helper method grow() for this

Part 4 - Bounds Checking

- Throw IndexOutOfBoundsException with a meaningful message for invalid indices in get(), set(), remove(int), and add(int, T)
- Throw IllegalArgumentException in the constructor if initialCapacity < 0

Part 5 - Iterator Support (Bonus)

- Implement the Iterable<T> interface so your class works in a **for-each loop**
- Implement a private inner class DynamicArrayIterator that implements Iterator<T>
- Support hasNext(), next(), and optionally remove()

Constraints

- Do **not** use ArrayList, LinkedList, or any java.util collection internally
- You **may** use Arrays.copyOf() and Objects.equals()
- Use **generics** (T) throughout

Part 6 - Testing: Write a Main.java test class that verifies:

```
System.out.println("=== Basic Add & Get ===");
DynamicArray<String> list = new DynamicArray<>();
list.add("Alice");
list.add("Bob");
list.add("Charlie");
System.out.println(list.get(1));    // Bob
System.out.println(list);           // [Alice, Bob, Charlie]

System.out.println("\n=== Add at Index ===");
list.add(1, "Zara");
System.out.println(list.get(1));    // Zara
System.out.println(list);           // [Alice, Zara, Bob, Charlie]

System.out.println("\n=== Remove by Index ===");
list.remove(0);
System.out.println(list.get(0));    // Zara
System.out.println(list);           // [Zara, Bob, Charlie]

System.out.println("\n=== Remove by Object ===");
list.remove("Charlie");
System.out.println(list.contains("Charlie")); // false
System.out.println(list);           // [Zara, Bob]

System.out.println("\n=== Set ===");
list.set(0, "Mia");
System.out.println(list);           // [Mia, Bob]

System.out.println("\n=== indexOf / lastIndexOf ===");
list.add("Bob");
System.out.println(list.indexOf("Bob")); // 1
System.out.println(list.lastIndexOf("Bob")); // 2

System.out.println("\n=== Resizing (20 elements into capacity 3) ===");
```

```

DynamicArray<Integer> nums = new DynamicArray<>(3);
for (int i = 0; i < 20; i++) nums.add(i);
System.out.println("Size: " + nums.size()); // 20
System.out.println(nums);

System.out.println("\n=== toArray ===");
Object[] arr = list.toArray();
for (Object o : arr) System.out.print(o + " ");

System.out.println("\n\n=== Iterator (for-each) ===");
for (String s : list) System.out.println(s);

System.out.println("\n=== Clear & isEmpty ===");
list.clear();
System.out.println("Empty: " + list.isEmpty()); // true

System.out.println("\n=== Exception Handling ===");
try {
    list.get(0);
} catch (IndexOutOfBoundsException e) {
    System.out.println("Caught: " + e.getMessage());
}

try {
    new DynamicArray<>(-1);
} catch (IllegalArgumentException e) {
    System.out.println("Caught: " + e.getMessage());
}

```

2. [20 points] MyLinkedList Package

Implement a **doubly linked list** class `MyLinkedList<T>` in Java that mimics the behavior of Java's `LinkedList`, using **nodes** as the underlying structure. You will practice pointer/reference manipulation, generics, and iterator design.

Background: Unlike arrays, a **linked list** stores elements in **nodes**, where each node holds:

- A **data** value
- A **pointer to the next node**
- A **pointer to the previous node** (*doubly linked*)

Part 1 : Node Class: Create a **private inner class** `Node<T>` inside `MyLinkedList` with:

- T data
- `Node<T> next`
- `Node<T> prev`
- A constructor `Node(T data)`

Part 2 : Core Structure

Your `MyLinkedList<T>` class must have:

- A private `Node<T> head` — points to the first node
- A private `Node<T> tail` — points to the last node
- A private `int size`
- A no-argument constructor that initializes an empty list

Part 3 : Required Methods

Implement **all** of the following:

Method	Description
void addFirst(T element)	Inserts at the beginning
void addLast(T element)	Inserts at the end
void add(T element)	Same as addLast
void add(int index, T element)	Inserts at given index
T getFirst()	Returns first element
T getLast()	Returns last element
T get(int index)	Returns element at index
T set(int index, T element)	Replaces element, returns old value
T removeFirst()	Removes & returns first element
T removeLast()	Removes & returns last element
T remove(int index)	Removes & returns element at index
boolean remove(Object o)	Removes first occurrence of object
int size()	Returns number of elements
boolean isEmpty()	Returns true if size == 0
boolean contains(Object o)	Returns true if element exists
int indexOf(Object o)	Returns first index, or -1
int lastIndexOf(Object o)	Returns last index, or -1
void clear()	Removes all elements
Object[] toArray()	Returns array of all elements

Stack / Queue Methods (*Bonus Part A*)

Method	Description
void push(T element)	Same as addFirst (stack behavior)
T pop()	Same as removeFirst (stack behavior)
T peek()	Returns first without removing
void offer(T element)	Same as addLast (queue behavior)
T poll()	Same as removeFirst, returns null if empty

Part 4 : Private Helper

Implement a private helper: private Node<T> nodeAt(int index)

- Traverses from **head** if index < size/2, else from **tail** — for efficiency
- Used internally by get(), set(), add(int, T), remove(int)

Part 5 : Bounds Checking

- Throw `IndexOutOfBoundsException` with a meaningful message for invalid indices
- Throw `NoSuchElementException` in `getFirst()`, `getLast()`, `removeFirst()`, `removeLast()` if the list is empty

Part 6 : Iterator Support)

- Implement `Iterable<T>` so the class works in a **for-each loop**
- Implement a private inner class `MyIterator` that implements `Iterator<T>`
- Support `hasNext()`, `next()`, and `remove()`
- **Extra challenge (Bonus):** implement a `descendingIterator()` that traverses from tail to head

Constraints

- Do **not** use `ArrayList`, `LinkedList`, or any `java.util` collection internally
- Do **not** use arrays to store elements
- Use **nodes with prev/next pointers** only
- Use **generics (T)** throughout

Part 7: Testing

Write a `Main.java` that verifies all operations:

```
System.out.println("=== addFirst / addLast ===");
MyLinkedList<String> list = new MyLinkedList<>();
list.addLast("B");
list.addFirst("A");
list.addLast("C");
System.out.println(list);           // [A, B, C]

System.out.println("\n=== add at index ===");
list.add(1, "X");
System.out.println(list);           // [A, X, B, C]

System.out.println("\n=== get / set ===");
System.out.println(list.get(2));     // B
list.set(2, "Z");
System.out.println(list);           // [A, X, Z, C]

System.out.println("\n=== removeFirst / removeLast ===");
list.removeFirst();
list.removeLast();
System.out.println(list);           // [X, Z]

System.out.println("\n=== remove by index / object ===");
list.add("W");
list.remove(0);
System.out.println(list);           // [Z, W]
list.remove("W");
System.out.println(list);           // [Z]

System.out.println("\n=== indexOf / lastIndexOf / contains ===");
list.add("Z");
list.add("Y");
System.out.println(list.indexOf("Z")); // 0
System.out.println(list.lastIndexOf("Z")); // 1
System.out.println(list.contains("Y")); // true

System.out.println("\n=== getFirst / getLast ===");
System.out.println(list.getFirst()); // Z
System.out.println(list.getLast());  // Y

System.out.println("\n=== toArray ===");
```

```

Object[] arr = list.toArray();
for (Object o : arr) System.out.print(o + " ");
System.out.println();

System.out.println("\n=== Stack behavior (Bonus) ===");
MyLinkedList<Integer> stack = new MyLinkedList<>();
stack.push(1); stack.push(2); stack.push(3);
System.out.println(stack);           // [3, 2, 1]
System.out.println(stack.pop());      // 3
System.out.println(stack.peek());     // 2

System.out.println("\n=== Queue behavior (Bonus) ===");
MyLinkedList<Integer> queue = new MyLinkedList<>();
queue.offer(10); queue.offer(20); queue.offer(30);
System.out.println(queue);           // [10, 20, 30]
System.out.println(queue.poll());     // 10
System.out.println(queue);           // [20, 30]

System.out.println("\n=== for-each Iterator (Bonus) ===");
for (String s : list) System.out.print(s + " ");
System.out.println();

System.out.println("\n=== Descending Iterator (Bonus) ===");
Iterator<String> desc = list.descendingIterator();
while (desc.hasNext()) System.out.print(desc.next() + " ");
System.out.println();

System.out.println("\n=== clear / isEmpty ===");
list.clear();
System.out.println("Empty: " + list.isEmpty()); // true
System.out.println(list);                      // []

System.out.println("\n=== Exception Handling ===");
try {
    list.getFirst();
} catch (java.util.NoSuchElementException e) {
    System.out.println("Caught: " + e.getMessage());
}

try {
    list.get(5);
} catch (IndexOutOfBoundsException e) {
    System.out.println("Caught: " + e.getMessage());
}

```